

PHP OOP Concepts

1.What is OOP?

OOP stands for Object-Oriented Programming. Object Oriented programming is one of the most popular programming based on the concept of objects and classes.

1.1.How to create a Class?

```
class ClassName {  
  
    Properties  
  
    Methods  
}
```

-Multiple Class in a single file.

```
class Test {  
  
}  
  
class Something {  
  
}
```

1.2.Creating Class Object

```
class Test {  
  
    Properties  
  
    Methods  
  
}  
  
$obj = new Test( );
```

-Creating **Multiple Object** of **Single Class**

1.3.Accessing Class Properties

```
class Person {  
  
    public $name = "Aung Aung";  
    public $age = 25;  
  
}  
  
$obj = new Person();  
echo $obj->name;  
echo $obj->age;
```

1.4.Accessing Class Methods

```
class Person {  
  
    public $name = "Aung Aung";  
    public $age = 25;  
    public function eat(){  
        echo "Loteria Fried Chicken!";  
    }  
}
```

```
$obj = new Person();  
echo $obj->name;  
echo "<br>";  
echo $obj->eat();
```

-Method accessing with **parameter**

```
class Person {  
  
    public $name = "Aung Aung";  
    public $age = 25;  
    public function eat($shop){  
        echo "Loteria Fried Chicken!" . $shop;  
    }  
}
```

```
$obj = new Person();  
echo $obj->name;  
echo "<br>";  
echo $obj->eat(Hledan);
```

1.5.Method Overloading

```
class Person {  
  
    public $name = "Aung Aung";  
    public $age = 25;  
    public function eat($shop="Hledan"){  
        echo "Loteria Fried Chicken!" . $shop;  
    }  
}  
  
$obj = new Person();  
echo $obj->name;  
echo "<br>";  
echo $obj->eat();  
echo "<br>";  
echo $obj->eat("Innyar Lake");
```

1.6.Access Modifier

- Public (Anyone can access)
- Private (Only accessible inside class)
- Protected (Only accessible by extended class and subclass)

1.7.\$this Variable

The **\$this** keyword refers to the **current object**, and is only available **inside methods**.

```
class Person {  
  
    public $name = "Aung Aung";  
  
    public function changeName( ) {  
  
        $this->name= "Tun Tun";  
  
    }  
}
```

```
$obj = new Person();  
echo $obj->name;  
echo "<br>";
```

```
$anotherObj = new Person();  
echo $anotherObj->changeName();  
echo $anotherObj->name;
```

1.8.Getter And Setter

```
class Test {  
  
    public $num=777;  
  
}
```

```
$obj = new Test ();  
echo $obj->$num;
```

-Getter Example

```
class Demo {  
  
    private $num=777;  
  
    public function getData( ){  
  
        return $this->num;  
  
    }  
}
```

```
$demoObj = new Demo ();  
echo $demoObj->getData();
```

-Setter Example

```
class Demo {  
  
    private $num=777;  
  
    public function setData( $data){  
  
        $this->$num=$data;  
  
    }  
  
    public function getData( ){  
  
        return $this->num;  
  
    }  
}
```

```
$obj = new Demo();  
echo $obj->setData(234);  
  
echo $obj->getData();
```

1.9.Inheritance

```
class Member {  
    public $name;  
    public $age;  
    public $education;  
    public function friend() {  
  
    }  
}
```

```
class GoldMember {  
    public $name;  
    public $age;  
    public $education;  
    public $coin;  
    public function friend() {  
  
    }  
}
```

```
class DiamondMember {  
    public $name;  
    public $age;  
    public $education;  
    public $diamond;  
    public function friend() {  
  
    }  
}
```

-Inheritance Example

```
class Member {  
    public $name="Kyaw Kyaw";  
    public $age=27;  
    public $education="B.C.Sc";  
    public function friend() {  
        echo "This is friend method!";  
    }  
}
```

```
class GoldMember extends Member {  
    public $coin=25;  
}  
$obj = new GoldMember();  
echo $obj->name;  
echo $obj->age;  
$obj->friend();
```

```
class DiamondMember extends Member {  
    public $diamond = 5;  
}  
$obj = new GoldMember();  
echo $obj->name;  
echo $obj->age;  
$obj->friend();
```

1.10.Overriding

```
class Member {  
    public $name;  
    public $age;  
    public $education;  
    public function friend() {  
        echo " Something Code";  
    }  
}
```

```
class GoldMember {  
    public $name;  
    public $age;  
    public $education;  
    public $coin;  
    public function friend() {  
        echo " Over write something Code!"  
    }  
}
```

```
class DiamondMember {  
    public $name;  
    public $age;  
    public $education;  
    public $diamond;  
    public function friend() {  
        echo " Over write something Code!"  
    }  
}
```

-Overriding Example

```
class Member {  
    public $name="Kyaw Kyaw";  
    public $age=27;  
    public $education="B.C.Sc";  
    public function doSomething() {  
        echo "This is doSomething Method!";  
    }  
  
    public function friend() {  
        echo "This is friend method!";  
    }  
}  
  
class GoldMember extends Member {  
    public $coin=25;  
    public function friend() {  
        $this->name="Zaw Zaw";  
        echo "This is Overwrite" . $this->name;  
    }  
}  
  
$obj = new GoldMember();  
$obj->doSomething();  
echo "<br>";  
$obj->friend();
```

1.11.Parent Constructor Invoking

```
class Member {  
  
    public function __construct() {  
  
        echo " Parent constructor";  
  
    }  
}
```

```
class GoldMember extends Member{  
  
    public function __construct() {  
  
        parent::__construct();  
        echo " Child constructor"  
  
    }  
}
```

1.12.Using Trait

```
trait First {  
  
    public $name = "Kyaw Kyaw";  
  
    public function doSomething(){  
        echo "Do Something!";  
    }  
}  
  
trait Second {  
  
    public $age = 27;  
  
}  
  
class Member {  
  
    use First,Second;  
    public function somethingElse(){  
        echo "Member Method!";  
    }  
}  
  
$obj = new Member();  
echo $obj->name;  
echo $obj->age;  
$obj->doSomething();  
$obj->somethingElse();
```

1.13.Magic Methods

1.13.1.Constructor

-can access argument

```
class Test {  
  
    public function __construct(){  
        echo "This is construct method!";  
    }  
}  
  
new Test();
```

1.13.2.Destructor

-destructor start when class is finish and destroy

-can't access argument

```
class Test {  
    private $result = 0;  
    public function __construct(){  
        echo "This is construct method!";  
    }  
  
    public function sum($num1,$num2){  
        $this->result=$num1+$num2;  
    }  
  
    public function __destruct(){  
        echo "This is destruct method!" . $this->result;  
    }  
}  
  
$obj = new Test();  
$obj->sum(22,55);
```

1.13.3.Get

-class ထဲမှာ မရှိသော property တွေကို object ကနေ လှမ်းပီး ခေါ်ယူ အသုံးပြုတဲ့ အခါ get magic method ကနေ အလို အလျောက် အလုပ်လုပ်ပေးတာ။

```
class Test {  
    public function __get ($name){  
        echo "You are trying to get {$name} property that doesn't  
exit";  
    }  
}  
  
$obj = new Text();  
$obj->name;
```

1.13.4.Set

-class ထဲမှာ မရှိသော property တွေကို object ကနေ လှမ်းပီး ထည့်ဖို့ကြိုတားတဲ့ အခါ set magic method ကနေ အလို အလျောက် အလုပ်လုပ်ပေးတာ။

```
class Test {  
  
    public function __set ($var , $value){  
        echo "You are trying to set non exit property {$var} with  
value of {$value}";  
    }  
}  
  
$obj = new Text();  
$obj->name="Aung Aung";
```

1.13.5.Isset

```
class Test {  
    public function __isset($name){  
        echo "You are trying check the set or not of {$name}  
property";  
    }  
}  
  
$obj = new Test ();  
// echo isset($obj->name) ? "True" : "False";  
isset($obj->age);
```

1.13.6.Unset

```
class Test {  
    public function __unset($name){  
        echo "You are trying check the unset or not of {$name}  
property";  
    }  
}  
  
$obj = new Test ();  
unset($obj->age);
```

1.13.7.Call

-မရှိတဲ့ method တွေကို object ကနေ လှမ်းပြီး ခေါ်ဖို့ ကြိုးစားတဲ့ အခါ call magic method ကနေ အလို အလျောက် အလုပ်လုပ်ပေးတာ။

```
class Test {  
    public function __call($method,$values){  
        echo "Your are trying to call method {$method} with  
values of {$values}";  
    }  
}
```

```
class Test {  
    public function __call($method,$values){  
        echo "Your are trying to call method {$method} with  
values of “;  
        echo "<pre>”.print_r($values,true).”</pre>”;  
    }  
}  
$obj = new Test();  
$obj->hello(“Aung Aung”);
```

1.13.7.CallStatic

-မရှိတဲ့ static method တွေကို object ကနေ လှမ်းပြီး ခေါ်ဖို့ ကြိုးစားတဲ့ အခါ call static magic method ကနေ အလို အလျောက် အလုပ်လုပ်ပေးတာ။

```
class Test {  
    public function __callStatic($method,$values){  
        echo "Your are calling none exit static method {$method}  
with values of {$values}”;  
    }  
}
```

```
Test::hello();
```

```
Test::hello("Kyaw Kyaw","Tun Tun");
```

1.13.7.ToString and Invoke

-Class Object တစ်ခုကို တိုက်ရိုက် ထုတ်ကြည့်တဲ့အခါ (ToString)

-Class Object တစ်ခုကို method အနေနဲ့ တိုက်ရိုက် ထုတ်ကြည့်တဲ့အခါ (Invoke)

```
class Test {
```

```
    public function __toString( ){  
        return "This class have no properties or methods";  
    }
```

```
    public function __invoke( ){  
        return "You are trying to call Object as Method";  
    }
```

```
}
```

```
$obj = new Test();
```

```
echo $obj; // toString
```

```
echo $obj(); // invoke
```

Thank You